AWS HOST AUDIT & CLEANUP

DevOps • Linux • Docker • AWS IAM Security

PROBLEM

Managing an AWS EC2 server means monitoring system resources, Docker containers, disk usage, running processes, and IAM security. Without a proper audit, unused resources can consume storage, and outdated access keys or missing MFA can create security risks.

HOW I DEBUGGED IT

I built a Python-based audit tool to investigate the server step by step.

- Checked CPU, RAM, and disk usage. Reviewed Docker containers, images, networks, and volumes.
- Identified large files and reviewed long-running or idle processes.
- Audited IAM users, MFA status, access keys, and administrative permissions.

THE SOLUTION

EC2 → Audit Checks → Safe Cleanup → Reports

I built a Python audit tool with Dry-Run mode, audit logs, and JSON reports to identify issues and quickly review cleanup actions.

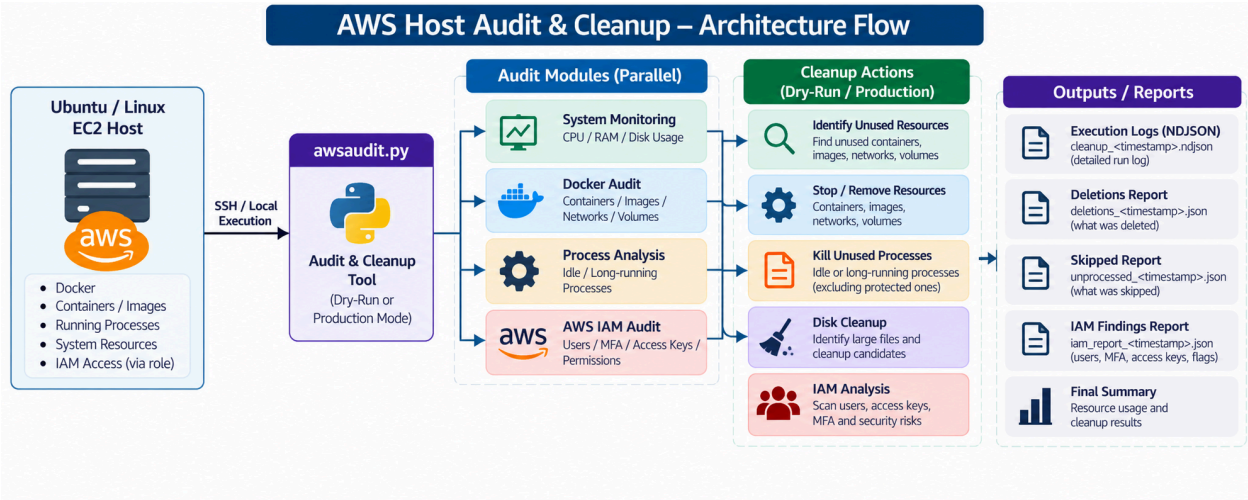
Has anyone else faced this problem?

How did you fix it?

Blog:https://vishnusai.hashnode.dev/aws-host-audit-cleanup?utm_source=hashnode&utm_medium=feed

HashnodProfile:https://hashnode.com/@vishnusaiK

GitHub : https://github.com/ksaivishnusaikvs/DevOps-Engineering/tree/main/Linkedin/Post




```
RESOURCE USAGE
CPU                                                0.0%
RAM                                                47.0%  686 MB / 1908 MB
Disk
  /                                                37%  7.2G / 19.7G
  /snap/amazon-ssm-agent/11797                     100%  0.0G / 0.0G
  /snap/amazon-ssm-agent/12322                     100%  0.0G / 0.0G
  /snap/core22/2163                                100%  0.1G / 0.1G
  /snap/btop/982                                   100%  0.0G / 0.0G
  /snap/core22/2411                                100%  0.1G / 0.1G
  /snap/core24/1499                                100%  0.1G / 0.1G
  /snap/snapd/26382                                100%  0.1G / 0.1G
  /snap/core24/1587                                100%  0.1G / 0.1G
  /snap/dust/19                                    100%  0.0G / 0.0G
  /snap/tree/54                                    100%  0.0G / 0.0G
  /snap/snapd/25935                                100%  0.1G / 0.1G
  /boot                                            20%  0.2G / 0.9G
  /boot/efi                                        6%  0.0G / 0.1G

ENTERPRISE CLEANUP MENU
Mode: DRY-RUN

  1  Full Auto     (Docker + Processes + Disk + IAM)
  2  Docker Only   (containers + images + nets + vols)
  3  Processes     (idle + long-running)
  4  Disk          (EBS + large files)
  5  IAM Tracker   (users, keys, MFA, inactive)
  6  Resources     (CPU / RAM / disk live)
  7  Exit

Select [1/2/3/4/5/6/7] (1):
```

IAM USER TRACKER

IAM Users Overview

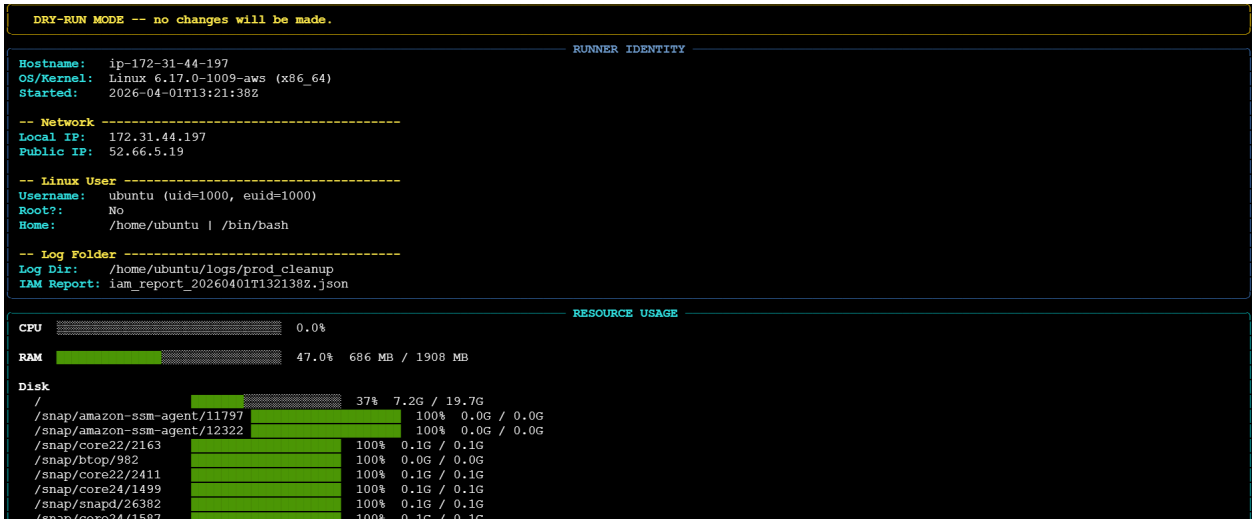
User	Console	MFA	Keys	Last Active	K1 Age(d)	K2 Age(d)	Admin	Groups	Flags
ROOT	YES	YES	none	3d ago	-	-	no	-	OK
cur-report-user	no	NO	none	never	-	-	no	-	OK
cur2-access-user	no	NO	K1	never	9	-	no	-	OK
devops-infra-cur-us…	no	NO	K1	14d ago	19	-	no	-	OK
infra-user	no	NO	K1	never	22	-	no	-	OK
readonelyacess	YES	NO	K1	3d ago	3	-	no	-	NO_MFA
readonly-user	no	NO	K1	never	4	-	no	-	OK
sahithi@710	YES	NO	K1 K2	41d ago	41	41	YES	-	NO_MFA ADMIN_ACCESS
sai	no	NO	K1 K2	2d ago	12	21	no	-	OK
vishnusai	YES	NO	K1 K2	0d ago	22	2	YES	-	NO_MFA ADMIN_ACCESS

3 users have security flags:

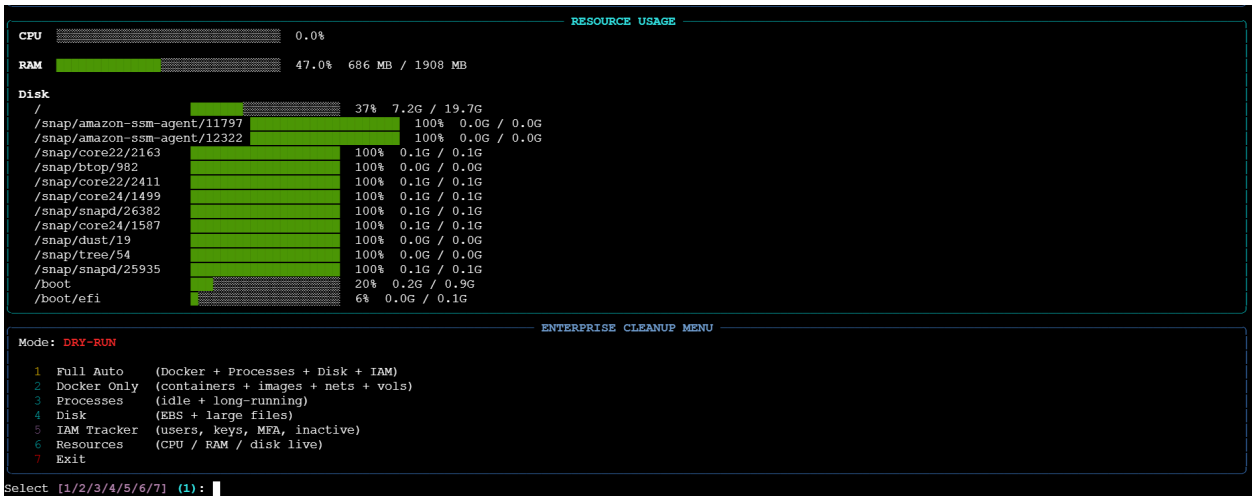
FLAGGED USERS

User	Flags	Policies	Recommendation
readonelyacess	NO_MFA	ReadOnlyAccess IAMUserChangePassword	Enable MFA immediately

Step1: System Monitoring: Displays server details, hostname, IP address, and logged-in user. Monitors CPU, RAM, and disk usage while running in Dry-Run mode without making changes.



Step2: The system resource screen provides a live view of CPU, memory utilization, and disk usage across mounted partitions. This helps identify servers or resources that are consuming high CPU, RAM, or disk capacity.



Step3: The document clearly shows Docker containers, images, networks, and volumes. However, the production cleanup is highly destructive because it can remove all containers, images, networks, and volumes. AWS MONITORING AUDIT(2)Add a clear warning before the production screenshot: “Production cleanup should be performed only after validating that the resources are unused.” sime and sho



ENTERPRISE CLEANUP MENU

Mode: DRY-RUN

1 Full Auto (Docker + Processes + Disk + IAM)
2 Docker Only (containers + images + nets + vols)
3 Processes (idle + long-running)
4 Disk (EBS + large files)
5 IAM Tracker (users, keys, MFA, inactive)
6 Resources (CPU / RAM / disk live)
7 Exit

Select [1/2/3/4/5/6/7] (1): 1

DOCKER FULL CLEANUP

No containers.
No images.

Networks

ID	Name	Driver
b6ea3707a138	ubuntu_default	bridge

No volumes.

DOCKER NUKE ONE-LINER

```
docker ps -aq | xargs -r docker stop && docker ps -aq | xargs -r docker rm -f && docker images -q | xargs -r docker rmi -f && docker network ls --filter type=custom -q | xargs -r docker network rm && docker volume ls -q | xargs -r docker volume rm && docker system prune -af --volumes  [DRY-RUN]
```

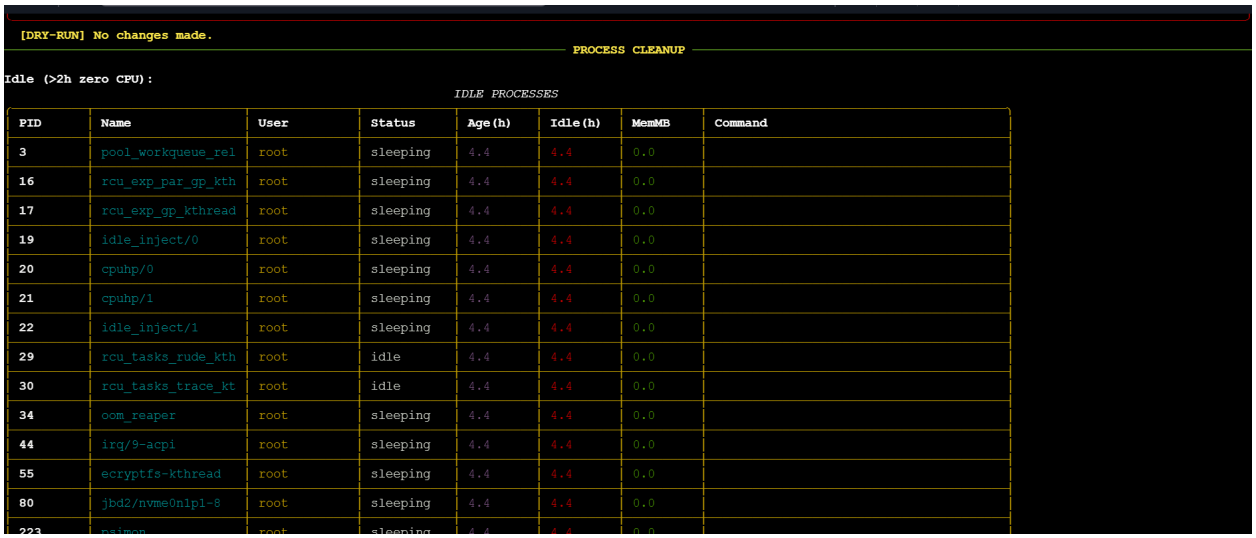
[DRY-RUN] No changes made.

PROCESS CLEANUP

Idle (>2h zero CPU):

IDLE PROCESSES

PID	Name	User	Status	Age(h)	Idle(h)	MemMB	Command
3	pool_workqueue_rel	root	sleeping	4.4	4.4	0.0	



[DRY-RUN] No changes made.

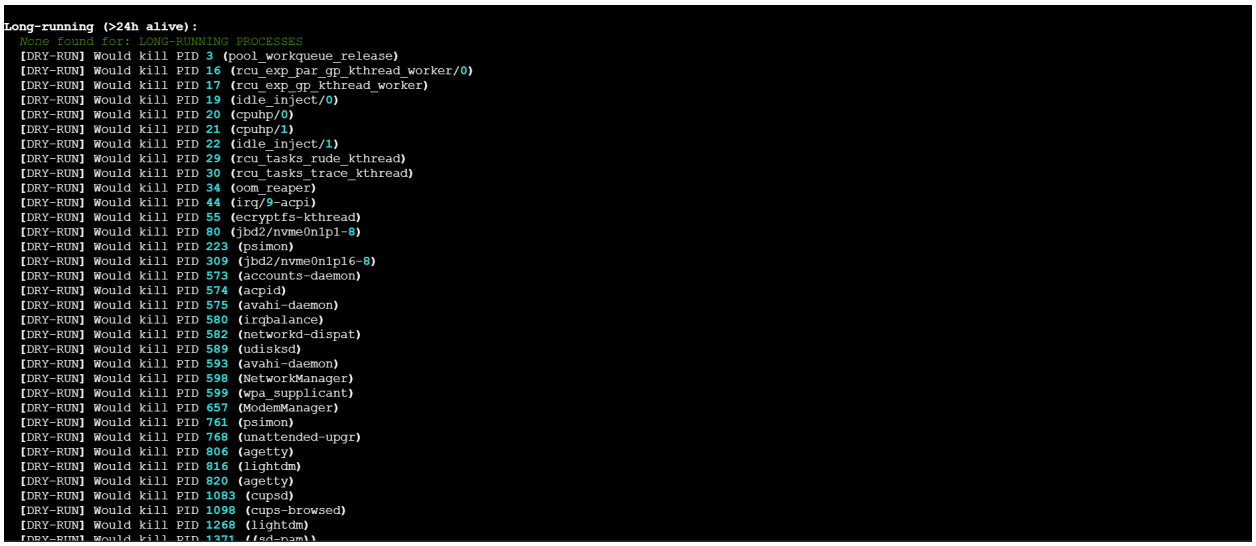
PROCESS CLEANUP

Idle (>2h zero CPU):

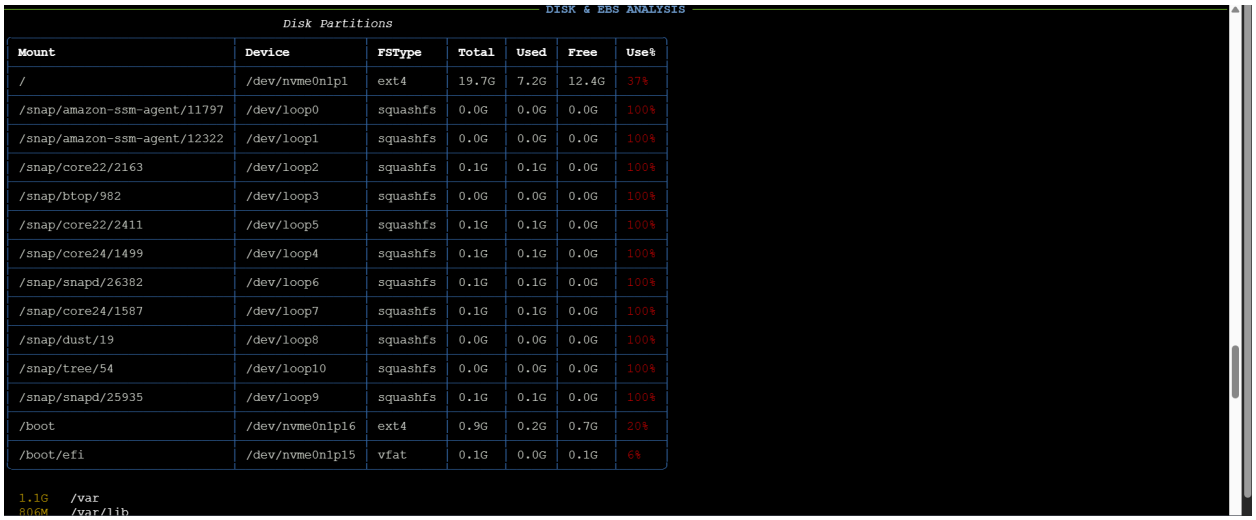
IDLE PROCESSES

PID	Name	User	Status	Age(h)	Idle(h)	MemMB	Command
3	pool_workqueue_rel	root	sleeping	4.4	4.4	0.0	
16	rcu_exp_par_gp_kth	root	sleeping	4.4	4.4	0.0	
17	rcu_exp_gp_kthread	root	sleeping	4.4	4.4	0.0	
19	idle_inject/0	root	sleeping	4.4	4.4	0.0	
20	cpuhp/0	root	sleeping	4.4	4.4	0.0	
21	cpuhp/1	root	sleeping	4.4	4.4	0.0	
22	idle_inject/1	root	sleeping	4.4	4.4	0.0	
29	rcu_tasks_rude_kth	root	idle	4.4	4.4	0.0	
30	rcu_tasks_trace_kt	root	idle	4.4	4.4	0.0	
34	oom_reaper	root	sleeping	4.4	4.4	0.0	
44	irq/9-acpi	root	sleeping	4.4	4.4	0.0	
55	ecryptfs-kthread	root	sleeping	4.4	4.4	0.0	
80	jbd2/nvme0n1p1-8	root	sleeping	4.4	4.4	0.0	
223	psimon	root	sleeping	4.4	4.4	0.0	

Step4: Good coverage of partitions, total/used/free space, and large files. The `/var`, `/tmp`, and `/home` analysis is useful for identifying storage-consuming files. AWS MONITORING AUDIT(2): Add the actual finding/result below the screenshot instead of only showing the command output.



Step5: The script checks idle and long-running processes and has a protected-process list. AWS MONITORING AUDIT(2): This is good from a safety perspective, but the document should explain why a process was identified as a cleanup candidate before showing termination.



/snap/tree/54	/dev/loop10	squashfs	0.0G	0.0G	0.0G	100%
/snap/snapd/25935	/dev/loop9	squashfs	0.1G	0.1G	0.0G	100%
/boot	/dev/nvme0n1p16	ext4	0.9G	0.2G	0.7G	20%
/boot/efi	/dev/nvme0n1p15	vfat	0.1G	0.0G	0.1G	6%

```
1.1G    /var
806M    /var/lib
493M    /var/lib/snapd
441M    /var/lib/snapd/snaps
409M    /home/ubuntu
409M    /home
253M    /home/ubuntu/jenkins
237M    /var/lib/apt
236M    /var/lib/apt/lists
186M    /home/ubuntu/jenkins/.git
184M    /home/ubuntu/jenkins/.git/objects
156M    /var/cache
150M    /var/log
144M    /var/log/journal/ec247f4b461d16df53113a4958e3e3fa
144M    /var/log/journal
137M    /home/ubuntu/gitkraken-amd64.deb
121M    /var/cache/apt
74M     /var/lib/snapd/snaps/core22_2411.snap
74M     /var/lib/snapd/snaps/core22_2163.snap
70M     /var/lib/apt/lists/ap-south-1.ec2.archive.ubuntu.com_ubuntu_dists_noble_universe_binary-amd64_Packages
```

Step6: This is one of the strongest sections. It checks **MFA, inactive users, access-key age, admin access, policies and groups**, and generates an IAM report. AWS MONITORING AUDIT(2): The recommendations such as MFA enablement, key rotation, and admin-access review are clearly presented.

3 users have security flags:

FLAGGED USERS

User	Flags	Policies	Recommendation
readonelyacess	NO_MFA	ReadOnlyAccess IAMUserChangePassword	Enable MFA immediately
sahithi@710	NO_MFA ADMIN_ACCESS	AdministratorAccess IAMUserChangePassword	Enable MFA immediately Audit admin necessity
vishnusai	NO_MFA ADMIN_ACCESS	AdministratorAccess IAMUserChangePassword	Enable MFA immediately Audit admin necessity

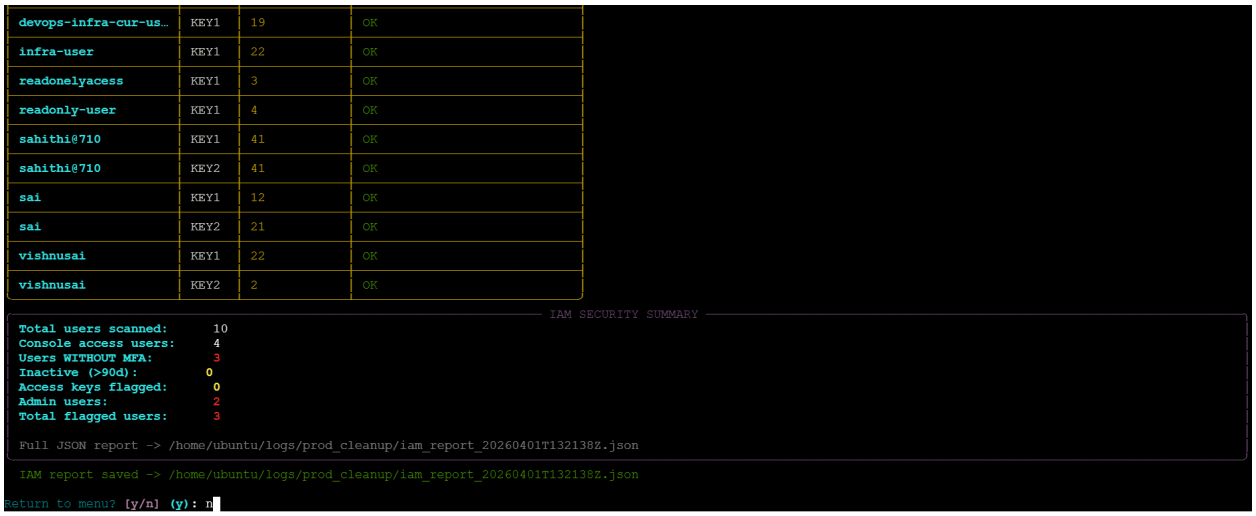
Active Access Keys

User	Key	Age (days)	Status
cur2-access-user	KEY1	9	OK
devops-infra-cur-us…	KEY1	19	OK
infra-user	KEY1	22	OK
readonelyacess	KEY1	3	OK
readonly-user	KEY1	4	OK
sahithi@710	KEY1	41	OK
sahithi@710	KEY2	41	OK
sai	KEY1	12	OK

Step7: IAM Security & Audit Logs:

The IAM audit reviews users, MFA, access keys, and flagged accounts, while recording audit activities.

The summary provides the security status and generates a detailed JSON audit report.



```
================================================================
ubuntu@ip-172-31-44-197:~$ cat awsaudit.py
#!/usr/bin/env python3
"""
╔═══════════════════════════════════════════════════════════════════════════════
═══════════╗
║       ENTERPRISE PRODUCTION CLEANUP SCRIPT  v4.0              ║
║  Docker · Processes · Disk · IAM User Tracker                 ║
║  Dry-Run + Production · Audit Trail · NDJSON Logs             ║
╚═══════════════════════════════════════════════════════════════════════════════
═══════════╝
"""

import os
import sys
import json
import signal
import socket
import datetime
import platform
import argparse
import subprocess
import textwrap
from pathlib import Path
from typing import List, Dict, Tuple, Optional

# ── Auto-install dependencies ──────────────────────────────────────────────
def _pip(pkg):
    subprocess.check_call([sys.executable, "-m", "pip", "install", pkg, "--quiet",
                           "--break-system-packages"], stderr=subprocess.DEVNULL)

try:
    from rich.console import Console
    from rich.table import Table
    from rich.panel import Panel
    from rich.prompt import Prompt, Confirm
    from rich.progress import Progress, SpinnerColumn, BarColumn, TextColumn,
TimeElapsedColumn
    from rich import box
    from rich.rule import Rule
except ImportError:
    _pip("rich")
    from rich.console import Console
    from rich.table import Table
```

```
    from rich.panel import Panel
    from rich.prompt import Prompt, Confirm
    from rich.progress import Progress, SpinnerColumn, BarColumn, TextColumn,
TimeElapsedColumn
    from rich import box
    from rich.rule import Rule

try:
    import psutil
except ImportError:
    _pip("psutil"); import psutil

try:
    import boto3
    from botocore.exceptions import ClientError, NoCredentialsError
except ImportError:
    _pip("boto3")
    import boto3
    from botocore.exceptions import ClientError, NoCredentialsError

#
═══════════════════════════════════════════════════════════════════════════════
═══════
# GLOBALS
#
═══════════════════════════════════════════════════════════════════════════════
═══════
console      = Console()
SCRIPT_START = datetime.datetime.now(datetime.timezone.utc)
LOG_DIR      = Path.home() / "logs" / "prod_cleanup"
RUN_TAG      = SCRIPT_START.strftime("%Y%m%dT%H%M%SZ")
LOG_DIR.mkdir(parents=True, exist_ok=True)

RUN_LOG       = LOG_DIR / f"cleanup_{RUN_TAG}.ndjson"
AUDIT_LOG     = LOG_DIR / "audit_master.ndjson"
DELETIONS_LOG = LOG_DIR / f"deletions_{RUN_TAG}.json"
UNPROC_LOG    = LOG_DIR / f"unprocessed_{RUN_TAG}.json"
IAM_REPORT    = LOG_DIR / f"iam_report_{RUN_TAG}.json"

DELETIONS  : List[dict] = []
UNPROCESSED: List[dict] = []
STATS: Dict = {
    "docker_containers_killed": 0,
    "docker_images_removed":    0,
```

```
    "docker_networks_removed":  0,
    "docker_volumes_removed":   0,
    "processes_killed":         0,
    "iam_users_scanned":        0,
    "iam_keys_flagged":         0,
    "iam_inactive_users":       0,
    "iam_no_mfa":               0,
}

PROTECTED_PROCS = {
    "init","systemd","kthreadd","kworker","ksoftirqd","migration","rcu_sched","rcu_bh",
    "watchdog","sshd","bash","python3","python","journald","dbus-daemon","rsyslogd",
    "chronyd","cron","atd","auditd","containerd","dockerd","docker","kubelet","etcd",
    "aws","amazon-ssm-agent","amazon-cloudwatch-agent","prod.py","prod_cleanup.py","cl.py",
}

IDLE_THRESHOLD_HOURS = 2
LONG_RUNNING_HOURS   = 24
KEY_WARN_DAYS        = 90
KEY_CRIT_DAYS        = 180
INACTIVE_DAYS        = 90

#
═══════════════════════════════════════════════════════════════════════════
═══════
# LOGGING HELPERS
#
═══════════════════════════════════════════════════════════════════════════
═══════
def _now_iso():
    return datetime.datetime.now(datetime.timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

def log_event(event_type, payload):
    entry = {"ts": _now_iso(), "type": event_type, **payload}
    for f in (RUN_LOG, AUDIT_LOG):
        with open(f, "a") as fp:
            fp.write(json.dumps(entry) + "\n")

def record_deletion(kind, ident, details=None):
    rec = {"ts": _now_iso(), "kind": kind, "id": ident, "details": details or {}}
    DELETIONS.append(rec); log_event("DELETE", rec)

def record_unprocessed(kind, ident, reason):
    rec = {"ts": _now_iso(), "kind": kind, "id": ident, "reason": reason}
```

```
    UNPROCESSED.append(rec); log_event("SKIP", rec)

def flush_json_logs():
    with open(DELETIONS_LOG, "w") as f: json.dump(DELETIONS, f, indent=2)
    with open(UNPROC_LOG,    "w") as f: json.dump(UNPROCESSED, f, indent=2)

#
═══════════════════════════════════════════════════════════════════════════
═══════
# UTILS
#
═══════════════════════════════════════════════════════════════════════════
═══════
def run(cmd, capture=True):
    r = subprocess.run(cmd, shell=True, capture_output=capture, text=True)
    return r.returncode, r.stdout.strip(), r.stderr.strip()

def docker_available():
    return run("docker info")[0] == 0

def bytes_to_mb(b):
    return round(b / 1_048_576, 2)

def bar(pct, width=30):
    f = int(pct / 100 * width)
    return "[green]" + chr(9608) * f + "[/green]" + chr(9617) * (width - f)

def days_ago(dt):
    if dt is None: return None
    if dt.tzinfo is None: dt = dt.replace(tzinfo=datetime.timezone.utc)
    return (datetime.datetime.now(datetime.timezone.utc) - dt).days

#
═══════════════════════════════════════════════════════════════════════════
═══════
# SECTION 1 — RUNNER IDENTITY
#
═══════════════════════════════════════════════════════════════════════════
═══════
def show_identity():
    hostname = socket.gethostname()
    uname    = platform.uname()
    try:    local_ip = socket.gethostbyname(hostname)
    except: local_ip = "unknown"
```

```
    rc, pub_ip, _ = run("curl -s --max-time 5 https://api.ipify.org")
    pub_ip = pub_ip if rc == 0 else "unavailable"
    try:
        import pwd
        pw = pwd.getpwuid(os.getuid())
        username, home, shell = pw.pw_name, pw.pw_dir, pw.pw_shell
    except:
        username = os.environ.get("USER","unknown")
        home, shell = os.path.expanduser("~"), "unknown"

    lines = [
        f"[bold cyan]Hostname:[/bold cyan]   {hostname}",
        f"[bold cyan]OS/Kernel:[/bold cyan]  {uname.system} {uname.release} ({uname.machine})",
        f"[bold cyan]Started:[/bold cyan]
{SCRIPT_START.strftime('%Y-%m-%dT%H:%M:%SZ')}",
        "",
        "[bold yellow]-- Network ---------------------------------------[/bold yellow]",
        f"[bold cyan]Local IP:[/bold cyan]   {local_ip}",
        f"[bold cyan]Public IP:[/bold cyan]  {pub_ip}",
        "",
        "[bold yellow]-- Linux User ------------------------------------[/bold yellow]",
        f"[bold cyan]Username:[/bold cyan]   {username} (uid={os.getuid()}, euid={os.geteuid()})",
        f"[bold cyan]Root?:[/bold cyan]      {'[bold red]Yes[/bold red]' if os.geteuid()==0 else 'No'}",
        f"[bold cyan]Home:[/bold cyan]       {home} | {shell}",
        "",
        "[bold yellow]-- Log Folder -------------------------------------[/bold yellow]",
        f"[bold cyan]Log Dir:[/bold cyan]    {LOG_DIR}",
        f"[bold cyan]IAM Report:[/bold cyan] {IAM_REPORT.name}",
    ]
    console.print(Panel("\n".join(lines), title="[bold]RUNNER IDENTITY[/bold]",
border_style="blue"))
    log_event("IDENTITY", {"hostname": hostname, "public_ip": pub_ip, "user": username})

#
═══════════════════════════════════════════════════════════════════════════════
═══════
# SECTION 2 — RESOURCE USAGE
#
═══════════════════════════════════════════════════════════════════════════════
═══════
def show_resource_usage():
    cpu = psutil.cpu_percent(interval=1)
    mem = psutil.virtual_memory()
    lines = [
```

```
        f"[bold]CPU[/bold]  {bar(cpu)}  {cpu:.1f}%\n",
        f"[bold]RAM[/bold]  {bar(mem.percent)}  {mem.percent:.1f}%  "
        f"{bytes_to_mb(mem.used):.0f} MB / {bytes_to_mb(mem.total):.0f} MB\n",
        "[bold]Disk[/bold]",
    ]
    for p in psutil.disk_partitions():
        try:
            u = psutil.disk_usage(p.mountpoint)
            lines.append(f"  {p.mountpoint:<20} {bar(u.percent, 20)}  "
                         f"{u.percent:.0f}%  {u.used/1e9:.1f}G / {u.total/1e9:.1f}G")
        except PermissionError:
            pass
    console.print(Panel("\n".join(lines), title="[bold]RESOURCE USAGE[/bold]",
border_style="cyan"))

#
═══════════════════════════════════════════════════════════════════════════
═══════
# SECTION 3 — DOCKER CLEANUP
#
═══════════════════════════════════════════════════════════════════════════
═══════
DOCKER_ONE_LINER = (
    "docker ps -aq | xargs -r docker stop && "
    "docker ps -aq | xargs -r docker rm -f && "
    "docker images -q | xargs -r docker rmi -f && "
    "docker network ls --filter type=custom -q | xargs -r docker network rm && "
    "docker volume ls -q | xargs -r docker volume rm && "
    "docker system prune -af --volumes"
)

def docker_cleanup(dry_run, interactive):
    if not docker_available():
        console.print("[yellow]  Docker daemon not reachable -- skipping.[/yellow]"); return
    console.print(Rule("[bold red]DOCKER FULL CLEANUP[/bold red]"))

    _, cr, _ = run("docker ps -a --format '{{.ID}}\t{{.Names}}\t{{.Status}}\t{{.Image}}'")
    _, ir, _ = run("docker images --format '{{.ID}}\t{{.Repository}}\t{{.Tag}}\t{{.Size}}'")
    _, nr, _ = run("docker network ls --filter type=custom --format '{{.ID}}\t{{.Name}}\t{{.Driver}}'")
    _, vr, _ = run("docker volume ls --format '{{.Name}}\t{{.Driver}}'")

    containers = [l.split("\t") for l in cr.splitlines() if l]
    images     = [l.split("\t") for l in ir.splitlines() if l]
    networks   = [l.split("\t") for l in nr.splitlines() if l]
```

```
volumes    = [l.split("\t") for l in vr.splitlines() if l]

def make_table(title, headers, rows, style="cyan"):
    t = Table(title=title, box=box.ROUNDED, border_style=style, show_lines=True)
    for h in headers: t.add_column(h, style="white")
    for r in rows:    t.add_row(*r[:len(headers)])
    return t

console.print(make_table("Containers", ["ID","Name","Status","Image"], containers)
              if containers else "[green]  No containers.[/green]")
console.print(make_table("Images",    ["ID","Repository","Tag","Size"], images, "yellow")
              if images     else "[green]  No images.[/green]")
console.print(make_table("Networks",  ["ID","Name","Driver"], networks, "magenta")
              if networks   else "[green]  No custom networks.[/green]")
if volumes:
    vt = Table(title="Volumes", box=box.ROUNDED, border_style="red")
    vt.add_column("Name"); vt.add_column("Driver")
    for v in volumes: vt.add_row(*v[:2])
    console.print(vt)
else:
    console.print("[green]  No volumes.[/green]")

console.print(Panel(
    f"[bold yellow]{DOCKER_ONE_LINER}{'  [DRY-RUN]' if dry_run else ''}[/bold yellow]",
    title="DOCKER NUKE ONE-LINER", border_style="red"))

if not (containers or images or networks or volumes):
    console.print("[green]  Docker already clean.[/green]"); return

if dry_run:
    console.print("[bold yellow]  [DRY-RUN] No changes made.[/bold yellow]")
    for c in containers: record_unprocessed("docker_container", c[0], "dry-run")
    for i in images:     record_unprocessed("docker_image",     i[0], "dry-run")
    return

if interactive and not Confirm.ask(
    f"[bold red]Remove {len(containers)} containers, {len(images)} images, "
    f"{len(networks)} networks, {len(volumes)} volumes?[/bold red]", default=False):
    console.print("[yellow]  Skipped.[/yellow]"); return

steps = [
    ("Stop all containers",    "docker ps -aq | xargs -r docker stop"),
    ("Remove all containers",  "docker ps -aq | xargs -r docker rm -f"),
    ("Remove ALL images",      "docker images -q | xargs -r docker rmi -f"),
```

```
        ("Remove custom networks", "docker network ls --filter type=custom -q | xargs -r docker
network rm"),
        ("Remove all volumes",     "docker volume ls -q | xargs -r docker volume rm"),
        ("System prune -af",       "docker system prune -af --volumes"),
    ]
    with Progress(SpinnerColumn(), TextColumn("[progress.description]{task.description}"),
                  BarColumn(), TimeElapsedColumn(), console=console) as prog:
        task = prog.add_task("Docker cleanup...", total=len(steps))
        for label, cmd in steps:
            prog.update(task, description=f"[cyan]{label}[/cyan]")
            rc, out, err = run(cmd)
            log_event("DOCKER_STEP", {"cmd": cmd, "rc": rc})
            prog.advance(task)

    console.print("[bold green]  Docker cleanup complete.[/bold green]")
    for c in containers:
        record_deletion("docker_container", c[0], {"name": c[1] if len(c)>1 else ""})
        STATS["docker_containers_killed"] += 1
    for i in images:
        record_deletion("docker_image", i[0]); STATS["docker_images_removed"] += 1
    for n in networks:
        record_deletion("docker_network", n[0]); STATS["docker_networks_removed"] += 1
    for v in volumes:
        record_deletion("docker_volume", v[0]); STATS["docker_volumes_removed"] += 1

#
═══════════════════════════════════════════════════════════════════════════════
═══════
# SECTION 4 — PROCESS CLEANUP
#
═══════════════════════════════════════════════════════════════════════════════
═══════
def _is_protected(name):
    nl = name.lower()
    return any(p in nl for p in PROTECTED_PROCS)

def _proc_age_h(proc):
    try: return (datetime.datetime.now().timestamp() - proc.create_time()) / 3600
    except: return 0.0

def _idle_h(proc):
    try:
        ct = proc.cpu_times()
        age = _proc_age_h(proc)
```

```
        return age if age > 0 and (ct.user + ct.system) < 0.5 else 0.0
    except: return 0.0

def collect_suspicious_processes():
    idle, long_r = [], []
    for proc in psutil.process_iter(["pid","name","username","status",
                                     "cpu_percent","memory_info","create_time","cmdline"]):
        try:
            info = proc.info
            name = info["name"] or ""
            if _is_protected(name) or info["pid"] in (0,1): continue
            if info["username"] in ("root", None) and name.startswith("k"): continue
            age_h = round(_proc_age_h(proc), 1)
            ih    = round(_idle_h(proc), 1)
            mem   = bytes_to_mb(info["memory_info"].rss) if info["memory_info"] else 0
            cmd   = " ".join(info["cmdline"] or [])[:120]
            entry = {"pid": info["pid"], "name": name, "user": info["username"] or "?",
                     "status": info["status"] or "?", "age_h": age_h,
                     "idle_h": ih, "mem_mb": mem, "cmd": cmd}
            if ih    > IDLE_THRESHOLD_HOURS: idle.append(entry)
            if age_h > LONG_RUNNING_HOURS:   long_r.append(entry)
        except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess): pass
    idle.sort(key=lambda x: x["idle_h"], reverse=True)
    long_r.sort(key=lambda x: x["age_h"], reverse=True)
    return idle, long_r

def show_process_table(title, procs, color="cyan"):
    if not procs:
        console.print(f"[green]  None found for: {title}[/green]"); return
    t = Table(title=title, box=box.ROUNDED, border_style=color, show_lines=True)
    for col, st, w in [("PID","bold white",8),("Name","cyan",18),("User","yellow",12),
                       ("Status","white",10),("Age(h)","magenta",8),("Idle(h)","red",8),
                       ("MemMB","green",8),("Command","dim",38)]:
        t.add_column(col, style=st, width=w)
    for p in procs[:50]:
        t.add_row(str(p["pid"]),p["name"][:18],p["user"][:12],p["status"],
                  str(p["age_h"]),str(p["idle_h"]),str(p["mem_mb"]),p["cmd"][:38])
    console.print(t)

def kill_processes(procs, dry_run, interactive, label):
    if not procs: return
    if interactive and not dry_run:
        console.print(f"\n[bold yellow]Found {len(procs)} {label} processes.[/bold yellow]")
        pids_input = Prompt.ask("PIDs to kill (comma-sep / all / none)", default="none")
```

```
        if pids_input.strip().lower() == "none":
            console.print("[dim]  Skipped.[/dim]"); return
        elif pids_input.strip().lower() == "all":
            targets = procs
        else:
            try:
                chosen  = {int(x.strip()) for x in pids_input.split(",")}
                targets = [p for p in procs if p["pid"] in chosen]
            except ValueError:
                console.print("[red]  Invalid -- skipping.[/red]"); return
    else:
        targets = procs

    killed = 0
    for p in targets:
        if dry_run:
            console.print(f"  [DRY-RUN] Would kill PID {p['pid']} ({p['name']})")
            record_unprocessed("process", str(p["pid"]), "dry-run"); continue
        try:
            os.kill(p["pid"], signal.SIGTERM)
            record_deletion("process", str(p["pid"]), {"name": p["name"]})
            STATS["processes_killed"] += 1; killed += 1
            console.print(f"  [green]  SIGTERM -> PID {p['pid']} ({p['name']})[/green]")
        except (ProcessLookupError, PermissionError) as e:
            console.print(f"  [red]  PID {p['pid']} -- {e}[/red]")
            record_unprocessed("process", str(p["pid"]), str(e))
    if not dry_run:
        console.print(f"[bold green]  Killed {killed}/{len(targets)}.[/bold green]")

def process_cleanup(dry_run, interactive):
    console.print(Rule("[bold yellow]PROCESS CLEANUP[/bold yellow]"))
    with console.status("[cyan]Scanning processes...[/cyan]"):
        idle_procs, long_procs = collect_suspicious_processes()
    console.print(f"\n[bold]Idle (>{IDLE_THRESHOLD_HOURS}h zero CPU):[/bold]")
    show_process_table("IDLE PROCESSES", idle_procs, "yellow")
    console.print(f"\n[bold]Long-running (>{LONG_RUNNING_HOURS}h alive):[/bold]")
    show_process_table("LONG-RUNNING PROCESSES", long_procs, "red")
    kill_processes(idle_procs,  dry_run, interactive, "idle")
    kill_processes(long_procs,  dry_run, interactive, "long-running")

#
═══════════════════════════════════════════════════════════════════════════
═══════
# SECTION 5 — DISK ANALYSIS
```

```
#
═══════════════════════════════════════════════════════════════════════════
═══════
def show_disk_analysis():
    console.print(Rule("[bold blue]DISK & EBS ANALYSIS[/bold blue]"))
    t = Table(box=box.ROUNDED, border_style="blue", title="Disk Partitions", show_lines=True)
    for col in ("Mount","Device","FSType","Total","Used","Free","Use%"):
        t.add_column(col, style="red" if col=="Use%" else "white")
    for p in psutil.disk_partitions(all=False):
        try:
            u = psutil.disk_usage(p.mountpoint)
            t.add_row(p.mountpoint, p.device, p.fstype,
                      f"{u.total/1e9:.1f}G", f"{u.used/1e9:.1f}G",
                      f"{u.free/1e9:.1f}G", f"{u.percent:.0f}%")
        except PermissionError: pass
    console.print(t)
    rc, out, _ = run("du -ahx --max-depth=4 /var /tmp /home 2>/dev/null | sort -rh | head -20")
    if out:
        lf = Table(box=box.SIMPLE, show_header=False)
        lf.add_column("size"); lf.add_column("path")
        for line in out.splitlines():
            parts = line.split(None, 1)
            if len(parts) == 2: lf.add_row(f"[yellow]{parts[0]}[/yellow]", parts[1])
        console.print(lf)

#
═══════════════════════════════════════════════════════════════════════════
═══════
# SECTION 6 — IAM USER TRACKER
#
═══════════════════════════════════════════════════════════════════════════
═══════
def _iam_client():
    try:
        c = boto3.client("iam")
        c.get_account_summary()
        return c
    except Exception:
        return None

def iam_tracker(dry_run):
    console.print(Rule("[bold magenta]IAM USER TRACKER[/bold magenta]"))

    iam = _iam_client()
```

```
if iam is None:
    console.print(Panel(
        "[yellow]No AWS credentials found.\n\n"
        "To enable IAM tracking:\n"
        "  export AWS_ACCESS_KEY_ID=...\n"
        "  export AWS_SECRET_ACCESS_KEY=...\n"
        "  export AWS_DEFAULT_REGION=ap-south-1\n\n"
        "Or attach an IAM role to this EC2 instance.[/yellow]",
        title="AWS Credentials Missing", border_style="yellow")); return

# Generate + fetch credential report
with console.status("[magenta]Requesting IAM credential report...[/magenta]"):
    import time
    for _ in range(30):
        resp = iam.generate_credential_report()
        if resp["State"] == "COMPLETE": break
        time.sleep(1)

csv_text = iam.get_credential_report()["Content"].decode("utf-8")
rows     = [r.split(",") for r in csv_text.strip().splitlines()]
headers  = rows[0]
users_csv= rows[1:]

def col(row, name):
    try:    return row[headers.index(name)]
    except: return ""

# Fetch all users, policies, groups
with console.status("[magenta]Listing users, policies, groups...[/magenta]"):
    all_users = []
    for page in iam.get_paginator("list_users").paginate():
        all_users.extend(page["Users"])

    policy_map = {}
    group_map  = {}
    for u in all_users:
        un = u["UserName"]
        attached = iam.list_attached_user_policies(UserName=un)["AttachedPolicies"]
        inline   = iam.list_user_policies(UserName=un)["PolicyNames"]
        policy_map[un] = [p["PolicyName"] for p in attached] + inline
        group_map[un]  = [g["GroupName"] for g in
                          iam.list_groups_for_user(UserName=un)["Groups"]]

STATS["iam_users_scanned"] = len(all_users)
```

```
def parse_dt(val):
    if val in ("N/A","no_information","","not_supported"): return None
    try:    return datetime.datetime.fromisoformat(val.replace("Z","+00:00"))
    except: return None

def key_age(raw):
    dt = parse_dt(raw)
    return days_ago(dt) if dt else None

enriched = []
for row in users_csv:
    uname = col(row, "user")
    if uname == "<root_account>": uname = "ROOT"

    pwd_last   = parse_dt(col(row, "password_last_used"))
    k1_last    = parse_dt(col(row, "access_key_1_last_used_date"))
    k2_last    = parse_dt(col(row, "access_key_2_last_used_date"))
    k1_age     = key_age(col(row, "access_key_1_last_rotated"))
    k2_age     = key_age(col(row, "access_key_2_last_rotated"))

    has_console = col(row, "password_enabled") == "true"
    has_mfa     = col(row, "mfa_active") == "true"
    k1_active   = col(row, "access_key_1_active") == "true"
    k2_active   = col(row, "access_key_2_active") == "true"

    pwd_days  = days_ago(pwd_last)
    k1_u_days = days_ago(k1_last)
    k2_u_days = days_ago(k2_last)

    last_active = min(
        (d for d in [pwd_days, k1_u_days, k2_u_days] if d is not None),
        default=None)

    policies = policy_map.get(uname, [])
    groups   = group_map.get(uname,  [])
    is_admin = "AdministratorAccess" in policies

    flags = []
    if has_console and not has_mfa:
        flags.append("NO_MFA"); STATS["iam_no_mfa"] += 1
    if last_active is not None and last_active > INACTIVE_DAYS:
        flags.append(f"INACTIVE_{last_active}d"); STATS["iam_inactive_users"] += 1
    if last_active is None and has_console:
```

```
            flags.append("NEVER_LOGGED_IN")
        if k1_active and k1_age is not None:
            if k1_age > KEY_CRIT_DAYS:
                flags.append(f"KEY1_CRITICAL_{k1_age}d"); STATS["iam_keys_flagged"] += 1
            elif k1_age > KEY_WARN_DAYS:
                flags.append(f"KEY1_WARN_{k1_age}d"); STATS["iam_keys_flagged"] += 1
        if k2_active and k2_age is not None:
            if k2_age > KEY_CRIT_DAYS:
                flags.append(f"KEY2_CRITICAL_{k2_age}d"); STATS["iam_keys_flagged"] += 1
            elif k2_age > KEY_WARN_DAYS:
                flags.append(f"KEY2_WARN_{k2_age}d"); STATS["iam_keys_flagged"] += 1
        if is_admin:
            flags.append("ADMIN_ACCESS")

        enriched.append({
            "user": uname, "console": has_console, "mfa": has_mfa,
            "k1_active": k1_active, "k2_active": k2_active,
            "k1_age": k1_age, "k2_age": k2_age,
            "last_active_d": last_active, "pwd_days": pwd_days,
            "policies": policies, "groups": groups,
            "is_admin": is_admin, "flags": flags,
        })

    # ── Table 1: All Users Overview ──────────────────────────────────────
    t_all = Table(title="IAM Users Overview", box=box.ROUNDED,
                  border_style="magenta", show_lines=True)
    t_all.add_column("User",         style="bold cyan",  width=20)
    t_all.add_column("Console",      style="white",      width=9)
    t_all.add_column("MFA",          style="white",      width=6)
    t_all.add_column("Keys",         style="white",      width=8)
    t_all.add_column("Last Active",  style="white",      width=13)
    t_all.add_column("K1 Age(d)",    style="yellow",     width=11)
    t_all.add_column("K2 Age(d)",    style="yellow",     width=11)
    t_all.add_column("Admin",        style="red",        width=7)
    t_all.add_column("Groups",       style="dim",        width=22)
    t_all.add_column("Flags",        style="red",        width=32)

    for u in enriched:
        mfa_s    = "[green]YES[/green]" if u["mfa"]     else "[red]NO[/red]"
        con_s    = "[green]YES[/green]" if u["console"] else "[dim]no[/dim]"
        keys_s   = f"{'K1 ' if u['k1_active'] else ''}{'K2' if u['k2_active'] else ''}".strip() or
"[dim]none[/dim]"
        act_s    = (f"{u['last_active_d']}d ago" if u["last_active_d"] is not None
                    else "[dim]never[/dim]")
```

```
    admin_s  = "[bold red]YES[/bold red]" if u["is_admin"] else "[dim]no[/dim]"
    groups_s = ", ".join(u["groups"][:3]) or "[dim]–[/dim]"
    flags_s  = " ".join(f"[red]{f}[/red]" for f in u["flags"]) or "[green]OK[/green]"
    t_all.add_row(u["user"], con_s, mfa_s, keys_s, act_s,
                  str(u["k1_age"]) if u["k1_age"] is not None else "–",
                  str(u["k2_age"]) if u["k2_age"] is not None else "–",
                  admin_s, groups_s, flags_s)
console.print(t_all)

# ── Table 2: Flagged / Risky Users ──────────────────────────────────
risky = [u for u in enriched if u["flags"]]
if risky:
    console.print(f"\n[bold red]  {len(risky)} users have security flags:[/bold red]")
    t_risk = Table(title="FLAGGED USERS", box=box.ROUNDED,
                   border_style="red", show_lines=True)
    t_risk.add_column("User",           style="bold cyan", width=20)
    t_risk.add_column("Flags",          style="red",       width=36)
    t_risk.add_column("Policies",       style="yellow",    width=32)
    t_risk.add_column("Recommendation", style="green",     width=38)

    for u in risky:
        recs = []
        if "NO_MFA"        in u["flags"]: recs.append("Enable MFA immediately")
        if any("INACTIVE"  in f for f in u["flags"]): recs.append("Disable or delete user")
        if any("CRITICAL"  in f for f in u["flags"]): recs.append("Rotate access keys NOW")
        if any("WARN"      in f for f in u["flags"]): recs.append("Schedule key rotation")
        if "ADMIN_ACCESS"  in u["flags"]: recs.append("Audit admin necessity")
        if "NEVER_LOGGED_IN" in u["flags"]: recs.append("Remove console access")
        t_risk.add_row(u["user"],
                       "\n".join(u["flags"]),
                       "\n".join(u["policies"][:5]) or "–",
                       "\n".join(recs))
    console.print(t_risk)
else:
    console.print("[green]  No security flags found.[/green]")

# ── Table 3: Access Keys Age ────────────────────────────────────────
key_users = [u for u in enriched if u["k1_active"] or u["k2_active"]]
if key_users:
    t_keys = Table(title="Active Access Keys", box=box.ROUNDED,
                   border_style="yellow", show_lines=True)
    t_keys.add_column("User",       style="bold cyan", width=20)
    t_keys.add_column("Key",        style="white",     width=5)
    t_keys.add_column("Age (days)", style="yellow",    width=12)
```

```
        t_keys.add_column("Status",     style="white",     width=28)

        for u in key_users:
            for key_n, active, age in [("KEY1",u["k1_active"],u["k1_age"]),
                                       ("KEY2",u["k2_active"],u["k2_age"])]:
                if not active: continue
                if age and age > KEY_CRIT_DAYS:
                    status = "[bold red]CRITICAL -- rotate immediately[/bold red]"
                elif age and age > KEY_WARN_DAYS:
                    status = "[yellow]WARNING -- rotate soon[/yellow]"
                else:
                    status = "[green]OK[/green]"
                t_keys.add_row(u["user"], key_n, str(age) if age else "?", status)
        console.print(t_keys)

    # ── IAM Summary Panel ──────────────────────────────────────────────────
    console.print(Panel(
        "\n".join([
            f"[bold cyan]Total users scanned:[/bold cyan]      {len(enriched)}",
            f"[bold cyan]Console access users:[/bold cyan]     {sum(1 for u in enriched if
u['console'])}",
            f"[bold cyan]Users WITHOUT MFA:[/bold cyan]        [bold
red]{STATS['iam_no_mfa']}[/bold red]",
            f"[bold cyan]Inactive (>{INACTIVE_DAYS}d):[/bold cyan]         [bold
yellow]{STATS['iam_inactive_users']}[/bold yellow]",
            f"[bold cyan]Access keys flagged:[/bold cyan]      [bold
yellow]{STATS['iam_keys_flagged']}[/bold yellow]",
            f"[bold cyan]Admin users:[/bold cyan]              [bold red]{sum(1 for u in enriched if
u['is_admin'])}[/bold red]",
            f"[bold cyan]Total flagged users:[/bold cyan]      [bold red]{len(risky)}[/bold red]",
            "",
            f"[dim]Full JSON report -> {IAM_REPORT}[/dim]",
        ]),
        title="IAM SECURITY SUMMARY", border_style="magenta"))

    # ── Save JSON report ───────────────────────────────────────────────────
    try:
        account_id = boto3.client("sts").get_caller_identity().get("Account","?")
    except Exception:
        account_id = "?"

    with open(IAM_REPORT, "w") as f:
        json.dump({"generated": _now_iso(), "account_id": account_id,
                   "users": enriched,
```

```
                "stats": {"total": len(enriched), "no_mfa": STATS["iam_no_mfa"],
                          "inactive": STATS["iam_inactive_users"],
                          "keys_flagged": STATS["iam_keys_flagged"],
                          "admin_users": sum(1 for u in enriched if u["is_admin"]),
                          "flagged": len(risky)}},
               f, indent=2, default=str)

    console.print(f"[green]  IAM report saved -> {IAM_REPORT}[/green]")
    log_event("IAM_SCAN", {"users": len(enriched), "flagged": len(risky)})

#
═══════════════════════════════════════════════════════════════════════════════
═══════
# SECTION 7 — FINAL SUMMARY
#
═══════════════════════════════════════════════════════════════════════════════
═══════
def show_summary(dry_run, elapsed):
    mode_tag = "[bold red]  DRY-RUN[/bold red]" if dry_run else "[bold green]
PRODUCTION[/bold green]"
    t = Table(box=box.DOUBLE_EDGE, border_style="green", title="CLEANUP SUMMARY",
show_lines=True)
    t.add_column("Metric",        style="bold cyan",  width=36)
    t.add_column("Count / Value", style="bold white", width=26)
    for k, v in [
        ("Mode",                      mode_tag),
        ("Docker containers killed",     str(STATS["docker_containers_killed"])),
        ("Docker images removed",        str(STATS["docker_images_removed"])),
        ("Docker networks removed",      str(STATS["docker_networks_removed"])),
        ("Docker volumes removed",       str(STATS["docker_volumes_removed"])),
        ("Processes killed",             str(STATS["processes_killed"])),
        ("IAM users scanned",            str(STATS["iam_users_scanned"])),
        ("IAM users without MFA",        str(STATS["iam_no_mfa"])),
        ("IAM inactive users",           str(STATS["iam_inactive_users"])),
        ("IAM access keys flagged",      str(STATS["iam_keys_flagged"])),
        ("Total duration",               f"{elapsed:.1f}s"),
        ("Audit log",                    str(AUDIT_LOG)),
        ("IAM report",                   str(IAM_REPORT)),
    ]: t.add_row(k, v)
    console.print(t)
    log_event("SUMMARY", {**STATS, "duration_s": elapsed, "dry_run": dry_run})
```

```
#
═══════════════════════════════════════════════════════════════════════
═══════
# INTERACTIVE MENU
#
═══════════════════════════════════════════════════════════════════════
═══════
MENU_ITEMS = [
    ("1", "Full Auto    (Docker + Processes + Disk + IAM)",   "auto"),
    ("2", "Docker Only  (containers + images + nets + vols)", "docker"),
    ("3", "Processes    (idle + long-running)",               "procs"),
    ("4", "Disk         (EBS + large files)",                 "disk"),
    ("5", "IAM Tracker  (users, keys, MFA, inactive)",        "iam"),
    ("6", "Resources    (CPU / RAM / disk live)",             "resources"),
    ("7", "Exit",                                             "exit"),
]
MENU_ACTION = {k: v for k, _, v in MENU_ITEMS}
MENU_COLORS = ["yellow","cyan","cyan","cyan","magenta","cyan","red"]

def interactive_menu(dry_run):
    mode_label = "[bold red]DRY-RUN[/bold red]" if dry_run else "[bold
green]PRODUCTION[/bold green]"
    body = f"Mode: {mode_label}\n\n" + "\n".join(
        f"  [{c}]{k}[/{c}]  {label}"
        for (k, label, _), c in zip(MENU_ITEMS, MENU_COLORS))
    console.print(Panel(body, title="[bold]ENTERPRISE CLEANUP MENU[/bold]",
border_style="blue"))
    return MENU_ACTION[Prompt.ask("Select", choices=[k for k,_,_ in MENU_ITEMS],
default="1")]

#
═══════════════════════════════════════════════════════════════════════
═══════
# MAIN
#
═══════════════════════════════════════════════════════════════════════
═══════
def main():
    # Declare ALL globals FIRST before any usage
    global IDLE_THRESHOLD_HOURS, LONG_RUNNING_HOURS
    global INACTIVE_DAYS, KEY_WARN_DAYS, KEY_CRIT_DAYS

    parser = argparse.ArgumentParser(
        description="Enterprise Cleanup + IAM Tracker v4.0",
```

```
    formatter_class=argparse.RawDescriptionHelpFormatter,
    epilog=textwrap.dedent("""
    Examples:
      sudo python3 prod_cleanup.py --dry-run
      sudo python3 prod_cleanup.py --production
      sudo python3 prod_cleanup.py --production --auto
      sudo python3 prod_cleanup.py --iam-only
      sudo python3 prod_cleanup.py --production --inactive-days 60 --key-crit 120
    """))

mode_grp = parser.add_mutually_exclusive_group()
mode_grp.add_argument("--dry-run",       action="store_true")
mode_grp.add_argument("--production",    action="store_true")
parser.add_argument("--auto",            action="store_true", help="No prompts (CI/CD)")
parser.add_argument("--docker-only",     action="store_true")
parser.add_argument("--procs-only",      action="store_true")
parser.add_argument("--disk-only",       action="store_true")
parser.add_argument("--iam-only",        action="store_true")
parser.add_argument("--idle-hours",      type=float, default=IDLE_THRESHOLD_HOURS)
parser.add_argument("--long-hours",      type=float, default=LONG_RUNNING_HOURS)
parser.add_argument("--inactive-days",   type=int,   default=INACTIVE_DAYS,
                    help="Flag IAM users inactive > N days (default 90)")
parser.add_argument("--key-warn",        type=int,   default=KEY_WARN_DAYS,
                    help="Access key warning age days (default 90)")
parser.add_argument("--key-crit",        type=int,   default=KEY_CRIT_DAYS,
                    help="Access key critical age days (default 180)")

args = parser.parse_args()

IDLE_THRESHOLD_HOURS = args.idle_hours
LONG_RUNNING_HOURS   = args.long_hours
INACTIVE_DAYS        = args.inactive_days
KEY_WARN_DAYS        = args.key_warn
KEY_CRIT_DAYS        = args.key_crit

dry_run     = not args.production
interactive = not args.auto

console.print(Panel(
    "[bold yellow]  DRY-RUN MODE -- no changes will be made.[/bold yellow]"
    if dry_run else
    "[bold red]  PRODUCTION MODE -- destructive operations WILL execute.[/bold red]",
    border_style="yellow" if dry_run else "red"))
```

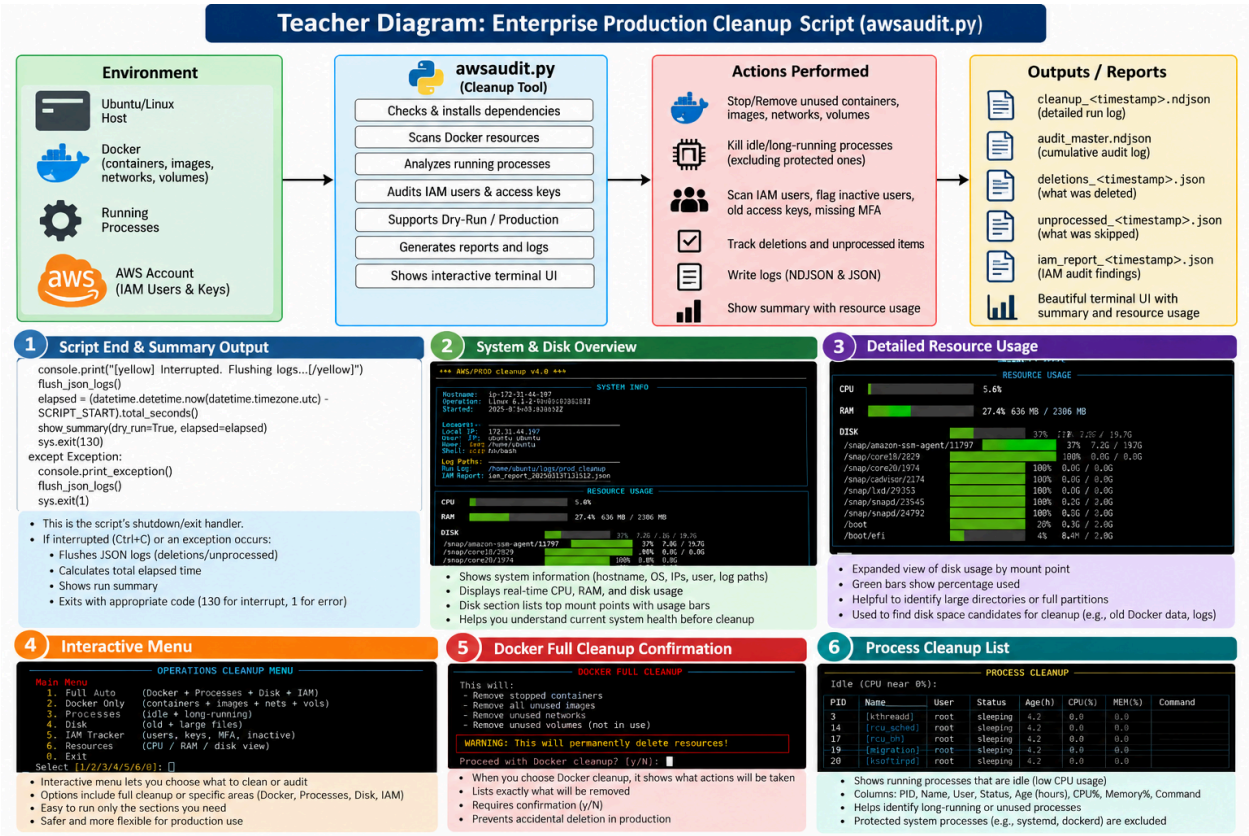
```
    show_identity()
    show_resource_usage()

    if args.iam_only:
        iam_tracker(dry_run)
    elif args.docker_only:
        docker_cleanup(dry_run, interactive)
    elif args.procs_only:
        process_cleanup(dry_run, interactive)
    elif args.disk_only:
        show_disk_analysis()
    elif args.auto:
        docker_cleanup(dry_run, interactive)
        process_cleanup(dry_run, interactive)
        show_disk_analysis()
        iam_tracker(dry_run)
    else:
        while True:
            action = interactive_menu(dry_run)
            if action == "exit": break
            elif action == "auto":
                docker_cleanup(dry_run, interactive)
                process_cleanup(dry_run, interactive)
                show_disk_analysis()
                iam_tracker(dry_run)
            elif action == "docker":    docker_cleanup(dry_run, interactive)
            elif action == "procs":     process_cleanup(dry_run, interactive)
            elif action == "disk":      show_disk_analysis()
            elif action == "iam":       iam_tracker(dry_run)
            elif action == "resources": show_resource_usage()
            if not Confirm.ask("\n[cyan]Return to menu?[/cyan]", default=True): break

    elapsed = (datetime.datetime.now(datetime.timezone.utc) - SCRIPT_START).total_seconds()
    flush_json_logs()
    show_summary(dry_run, elapsed)
    console.print(Panel(
        f"[bold green]  Completed![/bold green]  Logs -> [cyan]{LOG_DIR}[/cyan]",
        border_style="green"))


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
```

```
        console.print("\n[yellow]  Interrupted. Flushing logs...[/yellow]")
        flush_json_logs()
        elapsed = (datetime.datetime.now(datetime.timezone.utc) -
SCRIPT_START).total_seconds()
        show_summary(dry_run=True, elapsed=elapsed)
        sys.exit(130)
    except Exception:
        console.print_exception()
        flush_json_logs()
        sys.exit(1)
```



Teacher Diagram: Enterprise Production Cleanup Script (awsaudit.py)

Environment

- Ubuntu/Linux Host
- Docker (containers, images, networks, volumes)
- Running Processes
- AWS Account (IAM Users & Keys)

awsaudit.py (Cleanup Tool)

- Checks & installs dependencies
- Scans Docker resources
- Analyzes running processes
- Audits IAM users & access keys
- Supports Dry-Run / Production
- Generates reports and logs
- Shows interactive terminal UI

Actions Performed

- Stop/Remove unused containers, images, networks, volumes
- Kill idle/long-running processes (excluding protected ones)
- Scan IAM users, flag inactive users, old access keys, missing MFA
- Track deletions and unprocessed items
- Write logs (NDJSON & JSON)
- Show summary with resource usage

Outputs / Reports

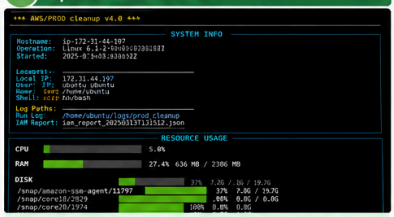
- cleanup_<timestamp>.ndjson (detailed run log)
- audit_master.ndjson (cumulative audit log)
- deletions_<timestamp>.json (what was deleted)
- unprocessed_<timestamp>.json (what was skipped)
- iam_report_<timestamp>.json (IAM audit findings)
- Beautiful terminal UI with summary and resource usage

1 Script End & Summary Output

```
    console.print("[yellow] Interrupted.  Flushing logs...[/yellow]")
    flush_json_logs()
    elapsed = (datetime.datetime.now(datetime.timezone.utc) -
    SCRIPT_START).total_seconds()
    show_summary(dry_run=True, elapsed=elapsed)
    sys.exit(130)
except Exception:
    console.print_exception()
    flush_json_logs()
    sys.exit(1)
```

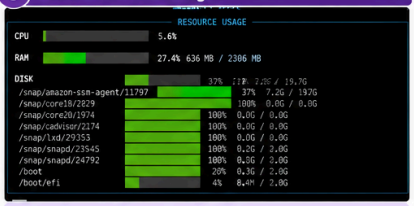
- This is the script's shutdown/exit handler.
- If interrupted (Ctrl+C) or an exception occurs:
  - Flushes JSON logs (deletions/unprocessed)
  - Calculates total elapsed time
  - Shows run summary
  - Exits with appropriate code (130 for interrupt, 1 for error)

2 System & Disk Overview



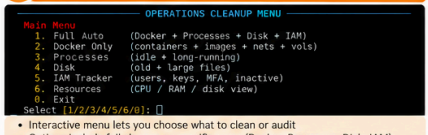
- Shows system information (hostname, OS, IPs, user, log paths)
- Displays real-time CPU, RAM, and disk usage
- Disk section lists top mount points with usage bars
- Helps you understand current system health before cleanup

3 Detailed Resource Usage



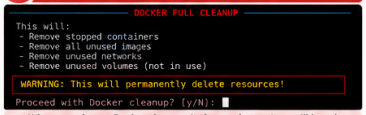
- Expanded view of disk usage by mount point
- Green bars show percentage used
- Helpful to identify large directories or full partitions
- Used to find disk space candidates for cleanup (e.g., old Docker data, logs)

4 Interactive Menu



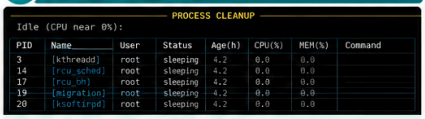
- Interactive menu lets you choose what to clean or audit
- Options include full cleanup or specific areas (Docker, Processes, Disk, IAM)
- Easy to run only the sections you need
- Safer and more flexible for production use

5 Docker Full Cleanup Confirmation



- When you choose Docker cleanup, it shows what actions will be taken
- Lists exactly what will be removed
- Requires confirmation (y/N)
- Prevents accidental deletion in production

6 Process Cleanup List



- Shows running processes that are idle (low CPU usage)
- Columns: PID, Name, User, Status, Age (hours), CPU%, Memory%, Command
- Helps identify long-running or unused processes
- Protected system processes (e.g., systemd, dockerd) are excluded

Teacher Diagram: Enterprise Production Cleanup Script (awsaudit.py)

Environment

- Ubuntu/Linux Host
- Docker (containers, images, networks, volumes)
- Running Processes
- AWS Account (IAM Users & Keys)

awsaudit.py (Cleanup Tool)

- Checks & installs dependencies
- Scans Docker resources
- Analyzes running processes
- Audits IAM users & access keys
- Supports Dry-Run / Production
- Generates reports and logs
- Shows interactive terminal UI

Actions Performed

- Stop/Remove unused containers, images, networks, volumes
- Kill idle/long-running processes (excluding protected ones)
- Scan IAM users, flag inactive users, old access keys, missing MFA
- Track deletions and unprocessed items
- Write logs (NDJSON & JSON)
- Show summary with resource usage

Outputs / Reports

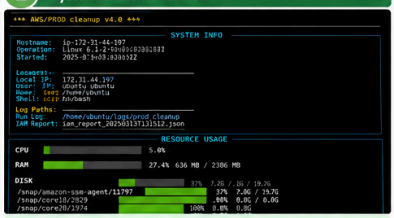
- cleanup_<timestamp>.ndjson (detailed run log)
- audit_master.ndjson (cumulative audit log)
- deletions_<timestamp>.json (what was deleted)
- unprocessed_<timestamp>.json (what was skipped)
- iam_report_<timestamp>.json (IAM audit findings)
- Beautiful terminal UI with summary and resource usage

1 Script End & Summary Output

```
    console.print("[yellow] Interrupted.  Flushing logs...[/yellow]")
    flush_json_logs()
    elapsed = (datetime.datetime.now(datetime.timezone.utc) -
SCRIPT_START).total_seconds()
    show_summary(dry_run=True, elapsed=elapsed)
    sys.exit(130)
except Exception:
    console.print_exception()
    flush_json_logs()
    sys.exit(1)
```

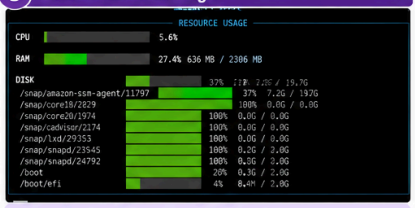
- This is the script's shutdown/exit handler.
- If interrupted (Ctrl+C) or an exception occurs:
  - Flushes JSON logs (deletions/unprocessed)
  - Calculates total elapsed time
  - Shows run summary
  - Exits with appropriate code (130 for interrupt, 1 for error)

2 System & Disk Overview



- Shows system information (hostname, OS, IPs, user, log paths)
- Displays real-time CPU, RAM, and disk usage
- Disk section lists top mount points with usage bars
- Helps you understand current system health before cleanup

3 Detailed Resource Usage



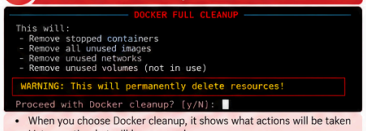
- Expanded view of disk usage by mount point
- Green bars show percentage used
- Helpful to identify large directories or full partitions
- Used to find disk space candidates for cleanup (e.g., old Docker data, logs)

4 Interactive Menu



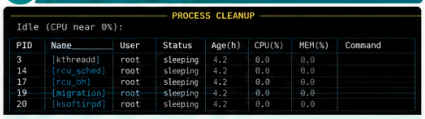
- Interactive menu lets you choose what to clean or audit
- Options include full cleanup or specific areas (Docker, Processes, Disk, IAM)
- Easy to run only the sections you need
- Safer and more flexible for production use

5 Docker Full Cleanup Confirmation



- When you choose Docker cleanup, it shows what actions will be taken
- Lists exactly what will be removed
- Requires confirmation (y/N)
- Prevents accidental deletion in production

6 Process Cleanup List



- Shows running processes that are idle (low CPU usage)
- Columns: PID, Name, User, Status, Age (hours), CPU%, Memory%, Command
- Helps identify long-running or unused processes
- Protected system processes (e.g., systemd, dockerd) are excluded